

Estudo Dirigido 04: Engenharia Reversa e Recuperação da Arquitetura de Software

Integrantes do grupo: Pedro Paiva Ferreira, Gabriel Nascimento Krepker, Joao Victor da Costa, Rafael Henriques Lima, Gabriel Monteiro, Mateus Silva Xavier

Sistema analisado: LUMS Library Management System, **versão refatorada** (com.biblioteca), resultado da refatoração feita pela equipe no ED03 a partir do código original. **Código-fonte:** Code/ (cópia local) e repositório original em github.com/Cozta1/ED03_Biblioteca_refatorada (pasta Refatorado/src/com/biblioteca)

1. Introdução

A Engenharia Reversa é a atividade de analisar um sistema existente, geralmente sem documentação confiável, para reconstruir o conhecimento sobre sua estrutura, seus componentes e seu comportamento. Um dos usos mais relevantes dessa prática é a **recuperação da arquitetura de software**. A partir do código-fonte, identificam-se classes, relacionamentos, módulos e padrões arquiteturais que descrevem como o sistema foi efetivamente construído.

Neste trabalho aplicamos essas técnicas sobre a **versão refatorada** do LUMS Library Management System (pacote com.biblioteca), produzida pela própria equipe no ED03 a partir da versão original sem pacotes. Aqui o objetivo é tratar esse código como um "sistema legado a investigar", como se a equipe não conhecesse suas decisões de design, e reconstruir, somente a partir do código-fonte, os diagramas e a descrição arquitetural correspondentes.

2. Descrição do Sistema

Objetivo do sistema: gerenciar o acervo e os usuários de uma biblioteca, permitindo cadastro de usuários (administrador, professor, estudante), cadastro de recursos (livros, pacotes de curso e revistas), empréstimo, devolução, solicitação de recursos indisponíveis e cálculo de multas por atraso.

Funcionalidades principais (extraídas de Main.java e do pacote ui):

- Login de Administrador ou Usuário (estudante ou professor) via MenuPrincipal
- Administrador (MenuAdmin chamando AdminService): criar usuários de qualquer tipo, criar livros, pacotes de curso e revistas, remover usuários e recursos
- Usuário (MenuUsuario chamando EmprestimoService): emprestar recurso, devolver recurso (com cálculo automático de multa), solicitar recurso indisponível, visualizar

recursos emprestados (ExibidorDeRecursos)

Tecnologias utilizadas: Java puro, sem frameworks externos, API moderna de datas (java.time.LocalDate, ChronoUnit) em vez da antiga Calendar/SimpleDateFormat, Streams (java.util.stream) para busca em coleções, Optional<T> para representar ausência de resultado, enum (TipoUsuario) em vez de constantes inteiras, e java.util.Scanner para entrada via console (CLI). Não há persistência em arquivo nem em banco de dados. O estado vive em memória durante a execução de Main.

3. Processo de Investigação

Como o sistema foi analisado: leitura estática de todos os 15 arquivos .java, organizados em 3 pacotes (model, service, ui) mais a classe Main na raiz (com.biblioteca), observando:

- as declarações package no topo de cada arquivo, para reconstruir a árvore real de pacotes (ao contrário da versão original, aqui ela existe fisicamente)
- as dependências de import entre pacotes, para inferir a direção das dependências (ui chama service que chama model)
- os construtores das classes de serviço (AdminService, EmprestimoService, MenuPrincipal, MenuAdmin, MenuUsuario), que recebem suas dependências via parâmetro, evidência de **Injeção de Dependência via construtor**
- o método main() em Main.java, único ponto onde os objetos são instanciados e conectados (Composition Root), para reconstruir o fluxo de inicialização do sistema

Técnicas utilizadas:

- Leitura de código-fonte assistida por busca textual (grep) por class, interface, enum, extends, package, import, new
- Reconstrução de diagrama de classes e de pacotes em **PlantUML**, a partir das evidências encontradas
- Comparação implícita com a versão original (sem pacotes) do mesmo domínio, o que ajudou a identificar quais decisões de design foram introduzidas na refatoração (camadas, enum, Optional, Streams, injeção de dependência)

Principais dificuldades encontradas:

- Identificar corretamente a direção das dependências entre service e ui, já que ambos manipulam os mesmos repositórios. Foi necessário conferir os imports de cada classe para confirmar que service nunca importa nada de ui
- Diferenciar associação de agregação entre RepositorioRecursos/RepositorioUsuarios e as listas internas (List<Recurso>,

List<Usuario>), já que a posse exclusiva do ciclo de vida dos objetos não é explícita apenas pelo tipo da coleção

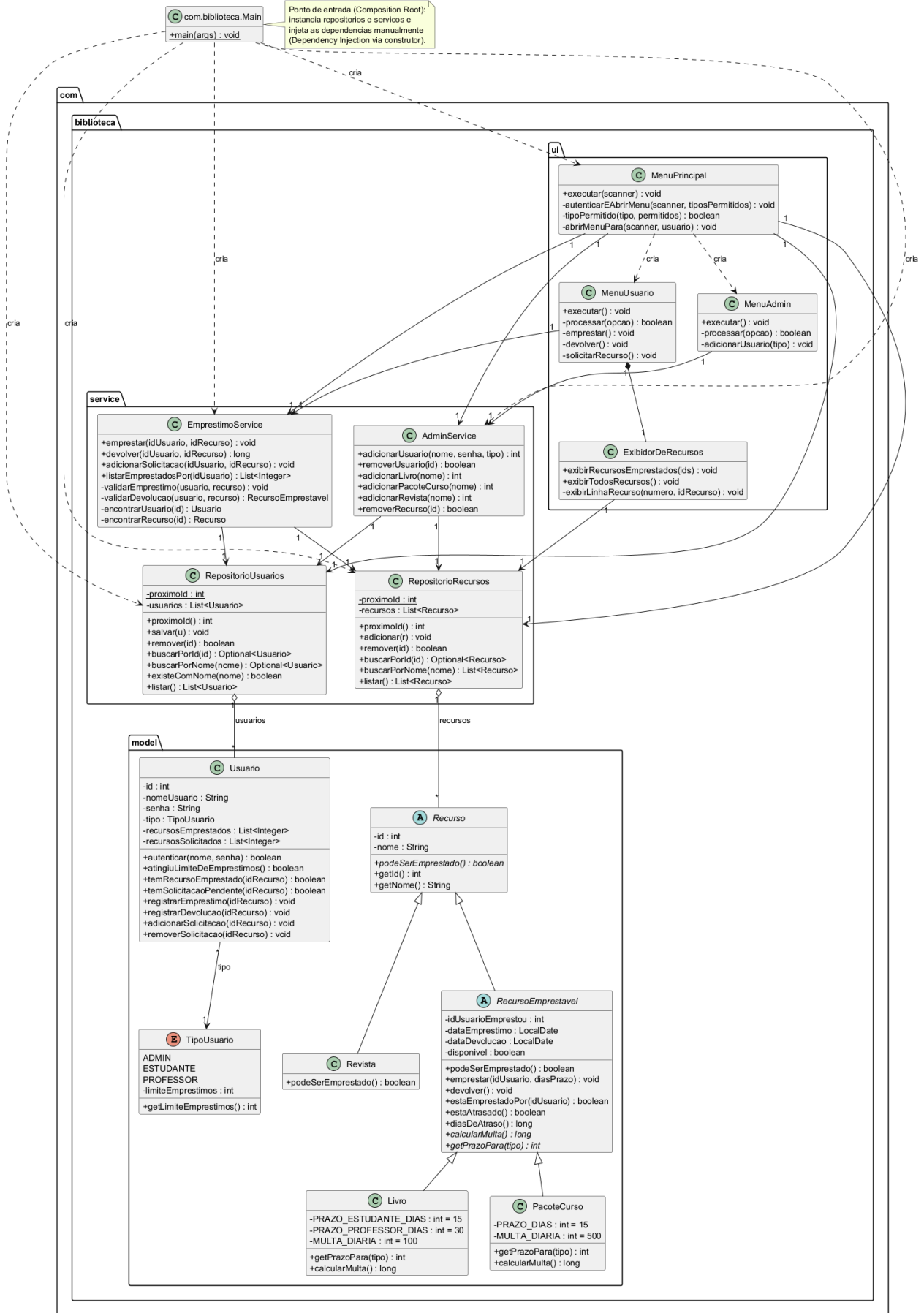
- Confirmar, por leitura de EmprestimoService, que o *cast* para RecursoEmprestavel ainda existe ((RecursoEmprestavel) recurso) mesmo após a refatoração, ou seja, nem todo acoplamento por tipo concreto foi eliminado

4. Arquitetura Recuperada

4.1 Diagrama de Classes

Arquivo-fonte: diagrams/class-diagram.puml

Diagrama de Classes - Sistema de Gerenciamento de Biblioteca (com.biblioteca, versao refatorada)

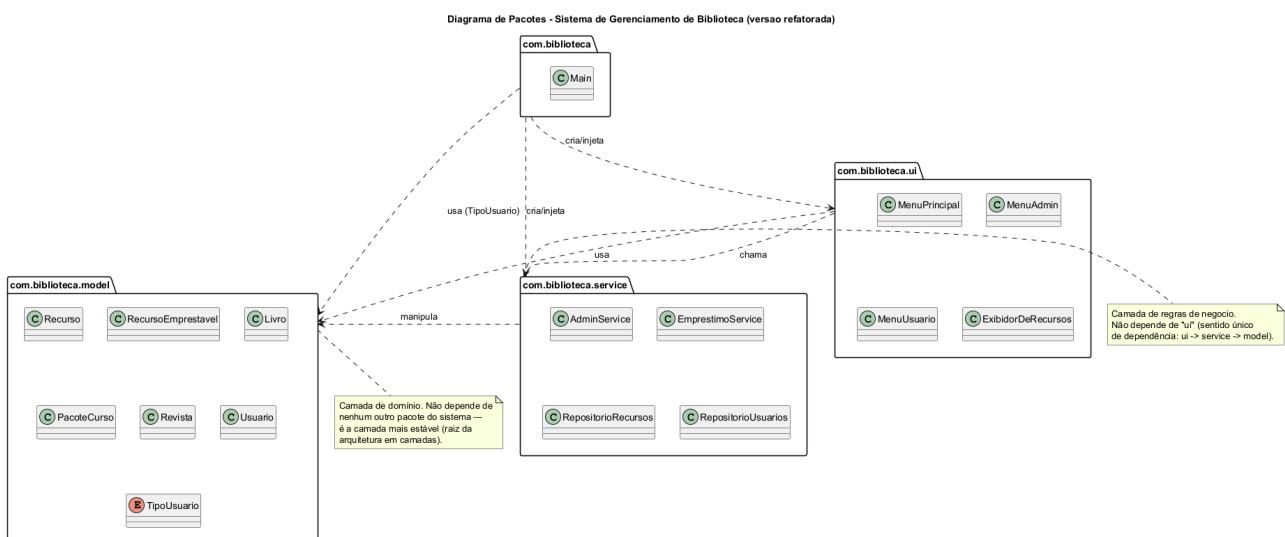


Principais elementos identificados:

- **Herança:** Recurso (abstrata) é especializada em RecursoEmprestavel (abstrata), que por sua vez é especializada em Livro e PacoteCurso. Recurso também é especializada diretamente em Revista
- **Enum:** TipoUsuario (ADMIN, ESTUDANTE, PROFESSOR), com regra de negócio embutida (limiteEmprestimos), substituindo os inteiros mágicos da versão original
- **Métodos abstratos e polimorfismo:** calcularMulta() e getPrazoPara(tipo) são sobrescritos por Livro e PacoteCurso, eliminando os condicionais por tipo que existiam na versão original
- **Associações:** AdminService e EmprestimoService usam RepositorioRecursos e RepositorioUsuarios (injetadas via construtor). Usuario referencia TipoUsuario
- **Agregações:** RepositorioRecursos agrega uma List<Recurso>. RepositorioUsuarios agrega uma List<Usuario>
- **Composição:** MenuUsuario cria e possui seu próprio ExibidorDeRecursos internamente, com ciclo de vida totalmente dependente do menu que o criou
- **Padrões de projeto identificados:** **Repository** (RepositorioRecursos, RepositorioUsuarios), **Service Layer** (AdminService, EmprestimoService) e **Dependency Injection via construtor**, com toda a árvore de objetos montada em Main.main, atuando como *Composition Root*

4.2 Estrutura de Pacotes

Arquivo-fonte: diagrams/package-diagram.puml



Ao contrário da versão original (sem pacotes), este código está fisicamente organizado em:

Pacote	Conteúdo	Papel arquitetural
com.biblioteca	Main	Composition root, ponto de entrada
com.biblioteca.model	Recurso, RecursoEmprestavel, Livro, PacoteCurso, Revista, Usuario, TipoUsuario	Domínio, entidades e regras intrínsecas a elas
com.biblioteca.service	AdminService, EmprestimoService, RepositorioRecursos, RepositorioUsuarios	Regras de negócio e acesso a dados em memória
com.biblioteca.ui	MenuPrincipal, MenuAdmin, MenuUsuario, ExibidorDeRecursos	Apresentação (CLI)

Dependências entre pacotes (unidirecionais): ui depende de service e de model. service depende de model. model não depende de nenhum outro pacote do sistema. Essa direção única de dependência, de fora para dentro, em direção ao domínio, é a evidência central de que o sistema segue uma **Arquitetura em Camadas (Layered Architecture)**.

4.3 Componentes e Responsabilidades

Componente	Classes	Responsabilidade observada
Apresentação (CLI)	MenuPrincipal, MenuAdmin, MenuUsuario, ExibidorDeRecursos	Interação via Scanner, formatação de saída, delegação para a camada de serviço, sem regra de negócio própria
Serviço (regras de negócio)	AdminService, EmprestimoService	Cadastro e remoção de usuários e recursos, validação de empréstimo, devolução e solicitação, cálculo de multa
Acesso a dados (em memória)	RepositorioRecursos, RepositorioUsuarios	CRUD simples sobre listas em memória, geração de IDs sequenciais

Componente	Classes	Responsabilidade observada
Domínio	Recurso, RecursoEmprestavel, Livro, PacoteCurso, Revista, Usuario, TipoUsuario	Estado e regras próprias de cada entidade (prazo, multa, limite de empréstimos)
Composição	Main	Monta o grafo de objetos e inicia a aplicação

4.4 Relacionamentos Encontrados (resumo)

- Main **cria e injeta** RepositorioRecursos e RepositorioUsuarios em AdminService e EmprestimoService, e injeta esses serviços em MenuPrincipal. Não há Singleton nem acesso estático global como na versão original
- MenuUsuario e MenuAdmin **dependem apenas da camada de serviço**, nunca acessam os repositórios diretamente (exceto ExibidorDeRecursos, que recebe RepositorioRecursos apenas para leitura e exibição)
- EmprestimoService ainda faz **cast explícito** (RecursoEmprestavel) recurso, um resquício de acoplamento ao tipo concreto que sobreviveu à refatoração
- Usuario **não conhece** Recurso, nem o contrário. A relação entre os dois é mediada inteiramente por EmprestimoService, reduzindo acoplamento direto entre entidades de domínio

5. Análise da Arquitetura

5.1 Padrão arquitetural identificado

O sistema segue uma **Arquitetura em Camadas (Layered Architecture)** de três camadas (ui, service e model), com dependências fluindo em uma única direção em direção ao domínio. Sobre essa base, identificam-se os padrões de projeto **Repository** (acesso a dados encapsulado em RepositorioRecursos/RepositorioUsuarios), **Service Layer** (AdminService e EmprestimoService concentram as regras de negócio) e **Dependency Injection via construtor**, com Main atuando como Composition Root. Não há indício de MVC propriamente dito, pois não existe um "Model" observável/observado nem um "Controller" formal. É mais preciso classificar como camadas técnicas simples.

5.2 Pontos fortes

- **Separação de responsabilidades clara:** a camada ui não contém regra de negócio (toda validação está em EmprestimoService/AdminService). A camada model não

conhece service nem ui

- **Eliminação do Singleton global:** em vez de `Library.getInstance()` acessado de qualquer lugar, como na versão original, o estado agora é injetado explicitamente, o que viabiliza testes unitários isolados
- **Polimorfismo no lugar de condicional por tipo:** `calcularMulta()` e `getPrazoPara(tipo)` são sobrescritos por subclasse, eliminando a lógica condicional repetida que existia em `Admin.addUser/addResource` na versão original
- **Uso de `enum` (`TipoUsuario`)** em vez de inteiros mágicos, tornando os tipos válidos auto-documentados e verificáveis em tempo de compilação
- **Uso de `Optional<T>`** em buscas (`buscarPorId`, `buscarPorNome`), evitando `NullPointerException` implícito presente na versão original, onde `findUser` podia retornar null silenciosamente
- **Tratamento de erro mais explícito:** `EmprestimoService` lança `IllegalStateException` e `IllegalArgumentException` com mensagens claras, capturadas e exibidas pela camada ui

5.3 Problemas identificados

- **Cast residual ao tipo concreto:** os métodos `emprestar` e `validarDevolucao` de `EmprestimoService` ainda fazem `(RecursoEmprestavel) recurso`. O acoplamento ao tipo concreto não foi totalmente substituído por polimorfismo, o que poderia ser resolvido com um método `tentarEmprestar()` no próprio `Recurso/RecursoEmprestavel`
- **Repositórios com estado estático (`proximold`):** `RepositorioRecursos` e `RepositorioUsuarios` usam um campo `static int proximold`, compartilhado entre todas as instâncias da classe. É um resquício do padrão de geração de ID global da versão original, que poderia causar colisão de IDs caso o sistema viesse a ter múltiplas instâncias de repositório, por exemplo em testes paralelos
- **`ExibidorDeRecursos`, na camada `ui`, depende diretamente de `RepositorioRecursos`**, que é da camada service, e não apenas do que `EmprestimoService` expõe. É uma pequena quebra da regra de "ui só fala com service", ainda que não viole a direção geral da dependência
- **Ausência de testes automatizados** no projeto (não há diretório test), apesar da arquitetura em camadas e da injeção de dependência facilitarem testes unitários
- **Sem persistência:** assim como na versão original, todo o estado é perdido ao encerrar o processo

5.4 Oportunidades de melhoria

- Mover a lógica de "tentar emprestar" e "tentar devolver" para dentro de `RecursoEmprestavel`, eliminando o cast em `EmprestimoService`

- Tornar proximoId um atributo de instância, ou um gerador de ID injetado, removendo o estado estático compartilhado dos repositórios
- Adicionar testes unitários para AdminService e EmprestimoService, aproveitando que ambos já recebem suas dependências via construtor
- Extrair uma interface comum (algo como Repository<T>) para RepositorioRecursos e RepositorioUsuarios, permitindo trocar a implementação em memória por uma com persistência real, sem alterar a camada service

6. Conclusão

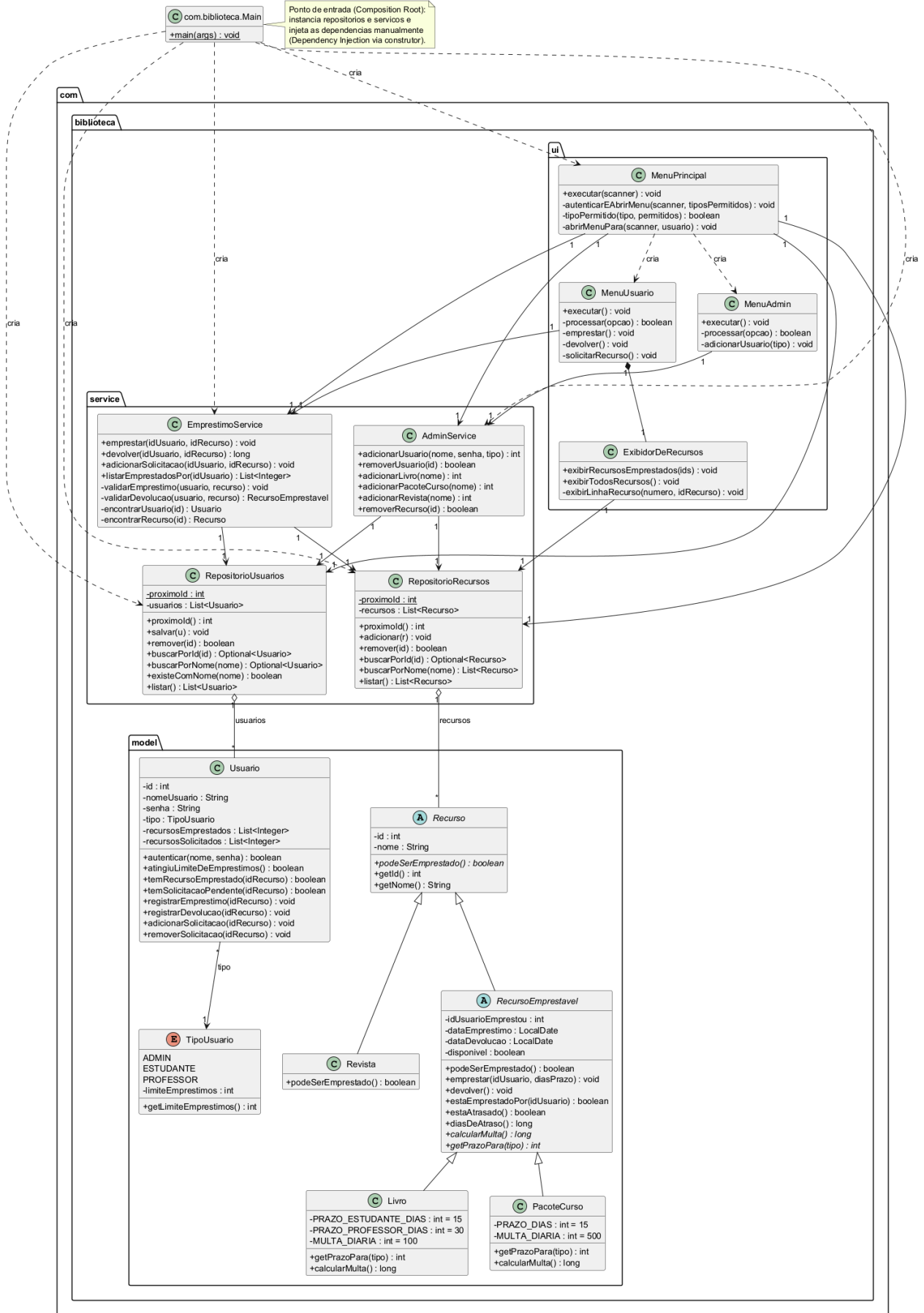
A análise da versão refatorada do LUMS Library System evidenciou uma arquitetura significativamente mais madura que a original: camadas físicas reais (model, service, ui), dependências unidirecionais, injeção de dependência via construtor e uso de recursos modernos da linguagem (enum, Optional, Streams, java.time). A engenharia reversa aqui serviu não apenas para descrever a estrutura, mas para **validar, a partir de evidências de código**, se a refatoração de fato alcançou os objetivos de separação de responsabilidades e baixo acoplamento. O resultado mostra que, embora a maior parte tenha sido alcançada, ainda restam pontos de acoplamento residual (cast a tipo concreto, estado estático nos repositórios) que não são visíveis apenas lendo a lista de pacotes.

O principal benefício percebido da Engenharia Reversa neste caso foi permitir uma **auditoria objetiva** da arquitetura, baseada em relações de dependência reais extraídas do código (imports, assinaturas de construtor), em vez de assumir que ter pacotes implica automaticamente em ter uma boa arquitetura em camadas. A maior dificuldade foi justamente diferenciar acoplamento aceitável, como uma camada chamando a camada interna seguinte, de acoplamento indevido, como resquícios de design antigo que sobreviveram à refatoração. Isso reforça a importância de revisar a arquitetura de tempos em tempos, mesmo depois de uma refatoração considerada concluída.

7. Ferramentas Utilizadas

- Leitura e análise de código: leitura manual assistida por busca textual sobre os arquivos .java
- Diagramas UML: **PlantUML** (diagrams/class-diagram.puml, diagrams/package-diagram.puml)

Diagrama de Classes - Sistema de Gerenciamento de Biblioteca (com.biblioteca, versao refatorada)



8. Entrega

- **Concluído:** Código-fonte analisado: Code/ e github.com/Cozta1/ED03_Biblioteca_refatorada
- **Concluído:** Diagramas UML (PlantUML): diagrams/
- **Concluído:** Relatório técnico: este documento (PDF e DOCX)